

Phase 3 Plan - Single Owner

Source of truth: Documentation/SRS.tex, Section 11.4, “Phase 3: Build Online State and Deterministic Candidate Detection”.

Execution status: Drafted for implementation kickoff after Phase 2 checkpoint.

Entry assumption: Phase 2 durable archives, detector-input logs, replay tooling, and local PostgreSQL runtime are available and academically defensible as the system of record.

Phase goal: Move from durable raw capture to stable online state, reproducible rolling features, and deterministic candidate episodes that can later feed evidence retrieval, replay validation, and ML ranking.

Interpretation note: Phase 3 is not a modeling phase and not an alert-delivery phase. It is the bridge between trustworthy replayable data and trustworthy candidate generation.

Implementation note: Every live feature or episode emitted in Phase 3 must be reproducible from replay using event-time semantics, watermark rules, and versioned detector logic.

Owned Deliverables (From SRS)

1. Stand up local Redis-backed rolling state for market, wallet, and event-family windows.
2. Implement watermark and late-event handling on event time rather than ingest time.
3. Build state reconciliation jobs that compare live rolling state against replay-rebuilt state.
4. Add throughput and memory monitoring for local Redis and detector workers.
5. Add persistent storage for candidate metadata, feature snapshots, and detector versions.
6. Define the first complete episode feature set for:
 - fresh-wallet count
 - fresh-wallet notional share
 - directional imbalance
 - concentration ratio
 - probability velocity
 - probability acceleration
 - volume acceleration
7. Implement deterministic candidate rules plus robust streaming statistics.
8. Define cooldown, suppression, and dedupe semantics for duplicate bursts.
9. Evaluate candidate quality on replayed historical windows and tune thresholds.
10. Produce one live candidate-generation service, one feature definition spec with schema version, one reconciliation report, and one Gate 3 signoff packet.

Phase 3 Exit Criteria

- Streaming features are stable and reconcile against replay within documented tolerances.
- Candidate generation is reproducible from replay.
- Detector state survives normal restarts without corrupting windows or episodes.
- Candidate rate is operationally bounded, explainable, and does not flood downstream operators.
- Every candidate persists feature snapshot, feature-schema version, detector version, triggering rules, and event-time window.

Locked Scope Decisions

- Phase 3 uses `event_time` as the canonical clock for windows and feature computation; `ingest_time` and `processing_time` are observability fields, not feature clocks.
- Redis is the online rolling-state engine for low-latency windows; PostgreSQL remains the durable system of record for persisted candidates, detector versions, and reconciliation outputs.
- Candidate generation remains deterministic rules plus robust rolling statistics only; no learned ranking model ships in Phase 3.
- External evidence retrieval, Telegram/Discord alerting, and analyst feedback are explicitly deferred to Phase 4.
- Deep models, graph neural networks, and sequence models remain out of scope.
- Feature windows must be replay-compatible from Phase 2 raw archives and normalized envelopes.
- Phase 3 may add durable schema for `signal_candidates`, `signal_episodes`, `signal_features`, and detector-version metadata, but it must not silently mutate Phase 2 archive semantics.

Concrete Artifacts Required

Workstream	Artifact	Done Condition
Online state runtime	Redis-backed state worker and configuration	Market, wallet, and event-family windows update continuously from detector-input envelopes
Time semantics layer	Watermark and late-event policy	Out-of-order events are incorporated or dropped according to a documented deterministic rule set
Feature contract	Versioned feature definition document	Every Phase 3 feature has a name, formula, window basis, null policy, and schema version
Candidate detector	Deterministic episode rules with threshold config	Candidate episodes emit from rolling features without manual intervention
Persistence layer	Durable candidate tables plus detector-version metadata	Each emitted candidate is queryable, versioned, and reproducible later
Reconciliation job	Replay-vs-live comparison workflow	Recent live windows can be rebuilt from replay and compared field by field within tolerance
Operational monitoring	Throughput, lag, memory, and candidate-rate metrics	Detector health can be observed during local runs and soak tests
Signoff evidence	Gate 3 report and reproducible validation outputs	The phase can be defended with concrete counts, tolerances, and example candidate windows

Required Persistent Contracts

1. Detector state contract

- Redis keys must be namespaced by environment and state type.

- Window state must exist at minimum for:
 - market
 - wallet
 - event family
- State entries must carry last processed `event_time`, last processed `ingest_time`, and a detector/state schema version.
- Restart behavior must prefer rebuilding short recent windows from durable detector-input logs rather than trusting partially written in-memory state.

2. Candidate persistence contract

- Every candidate row must persist:
 - candidate identifier
 - market identifier
 - event identifier
 - event-family identifier
 - episode start and end event time
 - trigger timestamp
 - feature snapshot payload
 - feature-schema version
 - detector version
 - triggering rules
 - cooldown/suppression decision state
- Candidate persistence must be append-friendly and auditable.
- Candidate rows must be replay-regenerable from raw archives plus detector version and feature version.

3. Reconciliation contract

- Reconciliation must compare live and replayed rolling features on the same event-time window.
- Tolerance must be explicit per feature family, especially for floating-point microstructure fields.
- Reconciliation outputs must record:
 - run identifier
 - replay window
 - detector version
 - feature-schema version
 - compared feature counts
 - mismatched rows
 - maximum absolute and relative error where relevant

Canonical Feature Set For Gate 3

Market microstructure features

- probability change
- probability velocity
- probability acceleration
- bid-ask spread
- depth imbalance
- realized volatility
- aggressive flow imbalance
- volume acceleration

Wallet and flow features

- first-seen age
- traded-footprint count
- first-trade notional
- fresh-wallet count by window
- fresh-wallet notional share
- concentration ratio
- repeated same-direction participation
- event-family overlap

Context features

- category
- time to resolution
- event popularity
- event-family breadth
- cross-market linkage strength
- market liquidity bucket

Gate 3 minimum rule-driving subset

- fresh-wallet count
- fresh-wallet notional share
- directional imbalance
- concentration ratio
- probability velocity
- probability acceleration
- volume acceleration

Deterministic Candidate Rules

The first detector version should stay simple, explainable, and replay-safe.

Rule family A: Fresh-wallet burst

- Trigger when fresh-wallet count and fresh-wallet notional share exceed configured thresholds inside a bounded event-time window.
- Require at least one supporting market-microstructure feature to exceed baseline, such as probability velocity or volume acceleration.
- Record whether the burst is same-direction concentrated or mixed.

Rule family B: Concentrated directional flow

- Trigger when directional imbalance and concentration ratio jointly exceed threshold.
- Require minimum notional and minimum participating-wallet count to avoid noise from tiny markets.
- Suppress duplicate candidates for the same market unless the new burst materially exceeds the previous trigger.

Rule family C: Fast repricing with wallet support

- Trigger when probability velocity or acceleration exceeds threshold and coincides with fresh-wallet or concentrated-flow evidence.
- Require the move to occur within a configured short window to preserve episode meaning.
- Record pre-window and post-window prices for later evaluation.

Robust statistics required

- rolling median and MAD
- exponentially weighted z-scores
- change-point style signals
- tail-event thresholds for rare bursts

Cooldown, Suppression, and Dedupe Policy

- A single market burst must not emit repeated candidates every time one more event lands in the same window.
- Cooldown must apply at minimum on:
 - market identifier
 - event-family identifier
 - trigger rule family
- A new candidate during cooldown is only allowed when severity materially increases according to a documented delta rule.
- Suppression decisions must be persisted so later replay can explain why an apparently qualifying window did or did not emit.
- Dedupe logic must distinguish:
 - same underlying episode seen multiple times
 - new episode after cooldown expiry
 - stronger continuation of the same episode

Concrete Execution Sequence

Block 1: Online state foundation

- Add Redis configuration and runtime wiring.
- Define state-key namespaces and window granularities for market, wallet, and event-family state.
- Build the detector-input consumer that updates rolling state from Phase 2 normalized envelopes.
- Record processing lag, watermark lag, and per-key update counts.

Block 2: Event-time correctness

- Define the canonical event-time extraction rules per source system.
- Implement watermark advancement and late-event handling.
- Decide and document when late events revise windows versus when they are counted as too late.
- Verify that restart and replay use the same event-time semantics as live processing.

Block 3: Feature contract v1

- Write the feature definition spec with schema version `phase3.v1`.
- Implement rolling computations for the Gate 3 minimum subset first.
- Add null/default policies and minimum-support conditions for illiquid windows.
- Persist feature snapshots for candidate-emitting windows.

Block 4: Deterministic detector v1

- Implement the first rule families and threshold configuration.
- Add cooldown, suppression, and dedupe semantics.
- Persist candidate rows and detector-version metadata.
- Measure candidate rate by market, category, and hour.

Block 5: Replay reconciliation

- Build a job that replays recent detector-input windows into Redis-equivalent state.
- Compare live and replayed rolling features field by field.
- Compare live and replayed candidate outputs.
- Write one reconciliation report with mismatch explanations and tolerance results.

Block 6: Gate 3 hardening

- Run a local soak session with monitoring enabled.
- Validate restart safety and short-window rebuild behavior.
- Tune thresholds so candidate volume is bounded and explainable.
- Assemble the Gate 3 signoff packet.

Files and Modules Expected To Change

- `config/settings.py` for Redis and detector configuration
- `database/schema.sql` and `database/postgres_schema.sql` for candidate and detector tables
- new detector/runtime modules for rolling state and candidate generation
- `run_collector.py` or a dedicated service entrypoint for the online detector worker
- `validation/` for live-vs-replay reconciliation tooling
- `ml_pipeline/feature_builder.py` only if it becomes the canonical batch materializer for Phase 3 feature reuse

Required Metrics and Monitoring

- detector-input envelopes processed per minute
- median and p95 processing lag
- median and p95 watermark lag
- Redis memory usage
- hot-key concentration and largest window states
- candidate count by hour
- candidate count by rule family
- suppressed-vs-emitted count
- replay reconciliation mismatch rate
- restart recovery duration

Gate 3 Evidence Pack Requirements

- one feature definition document with schema version
- one detector version record and threshold configuration snapshot
- one reconciliation report comparing live vs replayed features
- one candidate-rate summary over a non-trivial live window
- one restart-safety proof showing windows do not corrupt on normal restart
- one example packet showing a candidate with full feature snapshot and trigger explanation

Out of Scope For Phase 3

- learned ranking models
- external web/news or X evidence retrieval
- Telegram or Discord alert delivery
- analyst feedback workflow
- paper trading, backtesting reports, or empirical thesis claims
- graph clustering beyond fields needed to preserve future compatibility

Bottom-Line Assessment

Phase 3 is done when the repo can produce stable, replay-reproducible candidate episodes from live detector-input streams without depending on manual judgment, future information, or non-versioned state. If the system can rebuild recent windows from replay, reconcile live vs replayed features within tolerance, survive restart safely, and emit bounded explainable candidates, then the project is ready to enter Phase 4 with a trustworthy candidate engine rather than a brittle heuristic prototype.